

JavaScript Variables

[Back to JS page](#)

Table of Contents

- [JavaScript Variables](#)
 - [Variables are Data Containers](#)
 - [Example 1](#)
 - [JavaScript Identifiers](#)
 - [JavaScript Underscore \(_\)](#)
 - [Example 2](#)
 - [JavaScript Dollar Sign \(\\$\)](#)
 - [Example 3](#)
 - [Declare a Variable Using let](#)
 - [Example 4](#)
 - [Declare a Variable Using const](#)
 - [Example 5](#)
 - [One Statement, Many Variables](#)
 - [Example 6](#)
 - [JavaScript Data Types](#)
 - [Example 7](#)
 - [The Assignment Operator](#)
 - [Example 8](#)
 - [Document](#)
 - [Reference](#)

Variables are Data Containers

JavaScript variables are containers for data.

JavaScript variables can be **declared** in 4 ways: `let` , `const` , `var` (older), or automatically.

Variables are **labels** for data values and **containers** for storing data.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Variables</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Variables are Data Containers</h4>
<p id="demo"></p>

<script>
let carName = "Volvo";
document.getElementById("demo").innerHTML = carName;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

JavaScript Identifiers

Variables are identified with **names** called **identifiers**.

Names can be short like x, y, z or descriptive like age, sum, carName.

Rules for constructing names (identifiers):

- Names can contain letters, digits, underscores, and dollar signs.
- Names must begin with a letter.
- Names can also begin with `$` and `_`.
- Numbers are not allowed as the first character in names.
- Names are case sensitive (y and Y are different variables).
- Reserved words (like JavaScript keywords) cannot be used as names.

JavaScript Underscore (_)

JavaScript treats underscore as a letter. Identifiers containing `_` are valid variable names.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Variables</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Underscore (_)</h4>
<p id="demo"></p>

<script>
let _firstName = "John";
let _lastName = "Doe";
let _age = 30;
document.getElementById("demo").innerHTML = _firstName + " " + _lastName + " is " + _age;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

JavaScript Dollar Sign (\$)

JavaScript also treats a dollar sign as a letter. Identifiers containing `$` are valid variable names.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Variables</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Dollar Sign ($)</h4>
<p id="demo"></p>

<script>
let $ = "Hello World";
let $$$ = 2;
let $myMoney = 5;
document.getElementById("demo").innerHTML = $ + " " + ($ + $$$ + $myMoney);
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Declare a Variable Using let

Creating a variable in JavaScript is called **declaring** a variable.

You declare a JavaScript variable with the `let` keyword.

After the declaration, the variable has no value (technically it is `undefined`).

To **assign** a value to the variable, use the equal sign:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Variables</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Declare a Variable Using let</h4>
<p id="demo"></p>

<script>
let carName = "Volvo";
document.getElementById("demo").innerHTML = carName;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Declare a Variable Using const

Always use `const` if the value should not be changed.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Variables</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Declare a Variable Using const</h4>
<p id="demo"></p>

<script>
const price1 = 5;
const price2 = 6;
let total = price1 + price2;
document.getElementById("demo").innerHTML = "Total: " + total;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

One Statement, Many Variables

You can declare many variables in one statement.

Start the statement with `let` or `const` and separate the variables by **comma**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Variables</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>One Statement, Many Variables</h4>
<p id="demo1"></p>
<p id="demo2"></p>
<p id="demo3"></p>

<script>
let person = "John Doe", carName = "Volvo", price = 200;
document.getElementById("demo1").innerHTML = person;
document.getElementById("demo2").innerHTML = carName;
document.getElementById("demo3").innerHTML = price;
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

JavaScript Data Types

JavaScript variables can hold many **data types**: numbers, strings, objects and more.

Strings are text written **inside quotes**.

Numbers are written **without quotes**.

If you put a number in quotes, it will be treated as a text string.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Variables</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>JavaScript Data Types</h4>
<p id="demo1"></p>
<p id="demo2"></p>
<p id="demo3"></p>

<script>
let pi = 3.14;
let person = "John Doe";
let answer = 'Yes I am!';
document.getElementById("demo1").innerHTML = pi;
document.getElementById("demo2").innerHTML = person;
document.getElementById("demo3").innerHTML = answer;
</script>

</body>
</html>
```

Example 7

Result [View Example](#)

The Assignment Operator

In JavaScript, the equal sign (`=`) is an **assignment** operator, not an **equal to** operator.

In JavaScript, `x = x + 5` makes perfect sense: it assigns the value of `x + 5` to `x`.


```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Variables</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Assignment Operator</h4>
<p id="demo"></p>

<script>
let x = 10;
x = x + 5;
document.getElementById("demo").innerHTML = "Value of x = x + 5 is " + x;
</script>

</body>
</html>
```

Example 8

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Variables](#)